

FLEXBMS

System Overview

Home battery energy storage system

Document date	14 August 2026
System	96s LiFePO4 home BESS
Nominal capacity	314 Ah / approximately 96.5 kWh
Primary controller	STM32G491
Document status	Living architecture overview

This document explains the current FlexBMS system at architecture level. Schematics and source code remain authoritative for implementation details.

Contents

1. System level	3
1.1. System roles	3
1.2. Energy and information flow	3
2. Hardware	3
2.1. Battery assembly	3
2.2. Module boards	4
2.3. Master controller	4
2.4. High-voltage path	4
2.5. Physical communications	5
3. Software	5
3.1. STM32	5
3.2. HV supervisor	6
3.3. ESP32	6
3.4. Energy management system	10
3.5. Home Assistant integration	10
3.6. Companion application	10
4. Operating concept	11
4.1. Start-up	11
4.2. Normal operation	11
4.3. Faulted operation	11
5. Interfaces and ownership	11
6. Current project structure	12
7. Open topics	12

1. System level

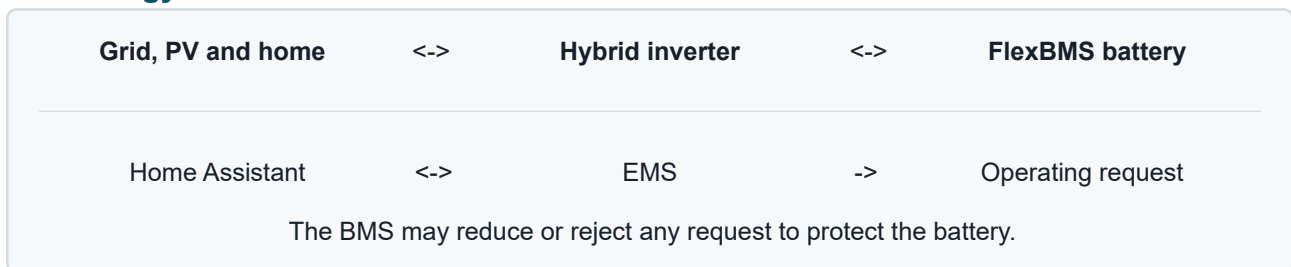
FlexBMS is the battery management and integration platform for a stationary home battery energy storage system. The battery consists of 96 LiFePO₄ cells in series, arranged as four 24-cell modules. At a nominal cell voltage of 3.2 V and a capacity of 314 Ah, the pack is approximately 307 V and 96.5 kWh.

The system connects the battery to a three-phase hybrid inverter. FlexBMS measures the battery, protects it, controls the high-voltage connection, and provides the information needed by higher-level energy management. The battery management system remains the final authority for battery safety.

1.1. System roles

Layer	Responsibility	Status
Battery and HV hardware	Stores energy, measures physical quantities, and safely connects the pack to the inverter.	CURRENT
STM32 BMS	Performs real-time measurement, protection, state handling, and contactor control.	CURRENT
ESP32 gateway	Provides local BMS maintenance, Wi-Fi provisioning, and telemetry transport while remaining listen-only on CAN.	CURRENT
Energy management system	Chooses when and how much to charge or discharge within BMS limits.	PLANNED
Home Assistant	Provides dashboards, automation, and a user-facing view of the installation.	PLANNED

1.2. Energy and information flow



Control principle

Energy management is advisory; battery protection is mandatory. The EMS may request an operating point, but the BMS defines the permitted charge and discharge envelope and can open the contactor.

2. Hardware

2.1. Battery assembly

The pack contains 96 series-connected LiFePO₄ cells divided into four physical 24s modules. Each module contains two twelve-cell monitoring sections. A 200 A / 75 mV shunt is located at the negative end of the pack for current measurement.

Item	Current baseline
Cell chemistry	LiFePO4
Series configuration	96s
Physical modules	4 x 24s
Cell capacity	314 Ah
Nominal pack voltage	307.2 V
Nominal stored energy	Approximately 96.5 kWh
Current shunt	200 A / 75 mV

The modules are installed in an unheated garage. Temperature measurement is distributed across the pack. The current STM32 configuration expects four NTC inputs per battery-monitor IC. Final sensor population and harness documentation should be kept consistent with the assembled modules.

2.2. Module boards

Each 24s module has one FlexBMS module board with two MC33771C battery cell controllers. Each controller measures twelve cells, supports passive balancing, reads local temperature sensors, and participates in the trans-former-isolated TPL daisy chain.

Across the complete pack this gives:

- eight MC33771C devices;
- 96 monitored cell voltages;
- up to 32 configured NTC measurements; and
- one current-sensing location on the first, lowest-voltage controller.

2.3. Master controller

The master board is the central interface between the battery, high-voltage switching hardware, service tools, inverter side, and local network.

Its main functional blocks are:

- STM32G491 real-time controller;
- MC33664 TPL transceiver for the module-board chain;
- ESP32-C3-WROOM-02U connectivity controller;
- isolated CAN and RS485 interfaces;
- isolated measurements of battery-side and load-side high voltage;
- contactor and precharge relay drivers;
- USB for flashing, diagnostics, and the Companion application; and
- power selection between USB and the battery-derived low-voltage supply.

The board contains separate HV, LV, and inverter-side isolated domains. USB-only power supports development and diagnostics, but does not provide the 12 V rail required to energize the contactor.

2.4. High-voltage path

The main contactor is located in the positive battery path. A separate precharge path limits inrush current before the contactor closes. The STM32 is the sole software owner of both drivers. It measures the battery and load sides through separate isolated AMC3330 channels, compares the battery-side result with the voltage reported by the cell-monitor chain, and observes both active-low diagnostic outputs.

The contactor is pulled in at full drive and then held with 20 kHz PWM. Precharge remains closed during pull-in and opens only after the contactor drive is established. The hold duty is a firmware configuration value.

Voltage feedback shows whether the two HV nodes are plausible and whether precharge has progressed. It does not prove the mechanical position of the contactor: the load-side voltage may remain present after opening because the inverter DC-link discharge behavior is not yet characterized.

Pack-level protection also includes the main DC fuse and a manual DC isolation device. Exact component ratings, wiring, creepage, clearance, and protection coordination belong in the hardware design files and commissioning documentation.

2.5. Physical communications

Interface	Connection	Purpose
TPL	Master to 8 monitor ICs	Cell, temperature, diagnostic, and current data.
USB	Service computer to STM32	Firmware, logs, and Companion access.
CAN	STM32 and inverter-side domain	Battery status and inverter communication.
Wi-Fi	ESP32 to local network	Local maintenance page, Wi-Fi provisioning, and future telemetry integration.
RS485	ESP32 to inverter-side equipment	Fallback or supplementary Modbus connection.

3. Software

The software is divided by responsibility. Time-critical protection and actuation remain on the STM32. Network-facing functions belong on the ESP32. Site optimization belongs in the EMS, with Home Assistant providing the local user and automation layer.

3.1. STM32

CURRENT

The STM32 firmware is the real-time BMS. It starts and monitors the MC33771C chain, gathers a complete battery measurement set, evaluates battery and communication faults, and controls relay permission.

The current battery profile is:

- eight MC33771C devices with twelve cells each;
- four NTC channels per controller on GPIO0 through GPIO3;
- current sensing on CID 1 with a 375 micro-ohm shunt value;
- positive current for charging and negative current for discharging;
- 314 Ah nominal capacity; and
- amp-hour retention in RTC backup registers while backup power is present.

The firmware separates active fault conditions, acknowledgement-required latches, and in-memory fault history. The HV supervisor receives permission only while the BMS is running without blocking faults and holds one fresh, complete measurement set from the monitor chain.

Runtime faults keep measurement and monitoring active while preventing relay closure. A clear request is accepted only after the underlying runtime condition has disappeared. The BMS then repeats its device, register, diagnostic, and measurement validation before allowing the latch to clear.

The STM32 also hosts:

- the dedicated HV supervisor;
- board-level analog and digital I/O;
- CAN communication;
- USB serial and the trusted text command/debug console; and
- the framed BMS v1 service interface over isolated UART and USB CDC.

3.1.1. Todo:

High priority:

- Go through CubeMX configuration. Pinout is OK.
- Watchdog implementation. Contactor should open in case of timeout.
- Analog measurement of HV side. Including diagnostic pin checking. ADC configuraion, high accuracy low Fs.
- Precharge handling. Including:
- Contactor handling. Including: economizing, failure checking using voltages (to be discussed)
- USB VSENSE setup
- Derating on temperature (mainly low temperature charging, high temperature is not that applicable)
- Communication to Goodwe inverter. Exact protocol needs discussion and likely imperical data once we have the inverter ready.

- User LED status language is specified in `Documentation/architecture/home-bess-firmware-and-maintenance.md`; firmware implementation remains pending.

Medium priority:

- Aux relay control (potentially useful for a future heating system).

3.2. HV supervisor

CURRENT

The HV supervisor follows OFF -> SELF_TEST -> PRECHARGE -> CONTACTOR_CLOSE -> RUN. The transport-independent `setRunRequest(bool)` operation is implemented through the framed BMS v1 named service. It provides intent only; it does not command a contactor. The STM32 accepts operation only when its BMS and HV-supervisor conditions are healthy.

SELF_TEST checks BMS permission, measurement freshness, isolated-voltage diagnostics, agreement between the battery-side and cell-monitor voltages, an unenergized load side, and the available power source. PRECHARGE raises the load-side voltage and requires a stable voltage relationship before contactor pull-in. CONTACTOR_CLOSE transfers from the precharge path to contactor hold drive. RUN keeps monitoring the BMS permission and both HV measurements.

A removed request, lost BMS permission, or implausible HV feedback disables both drivers. A blocking fault does not erase a true run request: after an explicit user fault-clear request and successful STM32 revalidation, the BMS may resume the startup sequence. The request defaults off after an STM32 reset; loss of a Gateway or USB Companion transport does not alter it. The STM32 reports requested state, actual HV state, and any blocking reason separately.

USB-only operation supports service access but cannot request or energize the HV path. The fitted hardware pull-down defines the inactive USB-sense level; the STM32 does not enable an internal pull.

Source of truth

Thresholds, timings, fault definitions, CAN identifiers, and register-level behavior are intentionally not duplicated here. The STM32 source code and hardware schematics are authoritative.

3.3. ESP32

CURRENT

The ESP32 is the network and telemetry controller in the isolated inverter-side domain. Its intended role is to observe BMS and inverter information, publish selected data to the local network, and support integration with the EMS and Home Assistant.

The ESP32 is not part of the battery safety chain. It is not allowed to transmit commands on the BMS CAN bus and must remain listen-only there. Loss of the ESP32 or Wi-Fi must not prevent the STM32 from protecting or isolating the battery.

Software/Gateway implements isolated UART v1, Wi-Fi provisioning/station/recovery, and the local compiled Companion maintenance page. The Companion provides stale-status diagnostics, fresh BMS monitoring, CSV logging, register reads, browser-time RTC synchronisation and device-time readback, fault clear, and immediate run-request actions. MQTT, OTA transfer, and CAN observation remain separate planned increments. The Gateway uses isolated UART to the STM32 and remains listen-only on shared BMS/GoodWe CAN. It is not a safety authority. The detailed transport, local-network, and recovery design is in `Documentation/architecture/home-bess-firmware-and-maintenance.md`.

The 4 MiB ESP32-C3 partition table uses two 1.5 MiB OTA application slots, a 512 KiB `bms_update` staging partition, and 384 KiB LittleFS reserved for future diagnostics/metadata. The committed Companion bundle is currently compiled into each application image, not served from LittleFS.

3.3.1. Framed BMS protocol v1 contract

AGREED

The canonical byte-level implementation contract is `Documentation/protocol/uart-v1.md`. The following overview is retained in the system document; resolve any discrepancy in favour of the Markdown contract.

The STM32G491 BMS and ESP32-C3 Gateway communicate over isolated USART1 at 1 Mbit/s, 8-N-1, full duplex, without hardware flow control. Direct USB CDC uses the same framed bytes for the Web Serial Companion while retaining the trusted text command/debug console through a byte dispatcher. The STM32 is the safety authority. UART loss is a communication-health condition only: it must not force the HV path off or change the current run request. The *! BMS Companion protocol is retired.

Each endpoint sends a zero-payload heartbeat every 500 ms. A peer is declared lost only after 1.5 s without a complete CRC-valid frame. Invalid bytes or frames with a bad CRC do not refresh the timer. The Gateway and direct USB Companion share one STM32 service slot; a second request receives BUSY rather than being queued.

All multi-byte values are little-endian. Encoders and decoders write and read individual fields; they must not transmit native C or C++ structs. The maximum payload length is 512 bytes.

3.3.1.1. Frame

offset	size	field
0	2	MAGIC = 0x46 0x42 ("FB")
2	1	VERSION = 0x01
3	1	TYPE
4	1	SEQUENCE
5	2	LENGTH, uint16 payload length
7	n	PAYLOAD
7+n	4	CRC32, uint32

CRC is CRC-32/ISO-HDLC: polynomial 0x04C11DB7 (reflected 0xEDB88320), reflected input and output, initial value and final xor 0xFFFFFFFF. It covers the magic, header, and payload, but not the CRC field; the transmitted CRC word is little-endian. The standard check value is $\text{CRC32}("123456789") = 0xCBF43926$. On an invalid version, length, or CRC, the receiver discards the first magic byte and resumes magic scanning. The protocol uses no escaping or byte stuffing.

SEQUENCE = 0 is reserved for heartbeat, telemetry, and events. A Gateway or USB Companion service request uses a value from 1 through 255; the STM32 response echoes that sequence.

3.3.1.2. Types

ID	Type	Direction
0x01	HEARTBEAT	both
0x02	STATUS	STM32 to Gateway or USB Companion
0x03	PACK	STM32 to Gateway or USB Companion
0x04	CELL	STM32 to Gateway or USB Companion
0x05	TEMPERATURE	STM32 to Gateway or USB Companion
0x10	SERVICE_REQUEST	Gateway or USB Companion to STM32
0x11	SERVICE_RESPONSE	STM32 to Gateway or USB Companion
0x12	EVENT	STM32 to Gateway or USB Companion

3.3.1.3. Telemetry payloads

HEARTBEAT has an empty payload.

STATUS is 17 bytes:

```
bms_state:u8 | hv_state:u8 | flags:u16 | slave_count:u8 |
bms_active_faults:u16 | bms_latched_faults:u16 |
hv_active_faults:u16 | hv_latched_faults:u16 | uptime_ms:u32
```

Status flag bits are: 0 BMS HV-ready, 1 charging allowed, 2 run request asserted, 3 complete measurements fresh, and 4 isolated-UART Gateway peer alive. USB Companion heartbeats do not affect bit 4. Bits 5–15 are zero. `slave_count` comes from the configured BMS monitor chain and is variable between builds. A production module contains two monitor slaves, but a single-slave development chain is supported until its second slave is connected. The Gateway must use the reported value and must not reject an odd count.

PACK is 24 bytes:

```
pack_voltage_uV:u32 | pack_current_raw:i16 | soc_raw:u16 |
min_cell_uV:u32 | max_cell_uV:u32 |
min_ntc_raw:u16 | max_ntc_raw:u16 | min_ic_raw:u16 | max_ic_raw:u16
```

Voltage values are microvolts. Current is signed amperes times 64; positive current is charging. SoC uses the existing BCC raw range (0 = -100%, 65535 = 200%), so percent is $100 * (soc_raw / 65535 * 3 - 1)$. NTC raw values convert as $raw / 65535 * 120 - 20$ degrees C. IC raw values are centikelvin, so degrees C are $raw / 100 - 273.15$.

CELL is 51 bytes, once per configured slave:

slave_index:u8 | balance_mask:u16 | cell_voltage_uV[12]:u32

slave_index is zero-based. module_index = slave_index >> 1 and slave_in_module = slave_index & 1. Balance-mask bit 0 represents cell 0 through bit 11 for cell 11; bits 12–15 are zero. Each slave always has twelve cells. In a single-slave development chain, only index 0 is sent; it maps to module 0, slave 0, and no partner frame is implied.

TEMPERATURE is 11 bytes, once per configured slave:

slave_index:u8 | ntc_raw[4]:u16 | ic_temp_raw:u16

The four NTC values and one IC value are included together, making this packet fixed-size. This matches the installed four-NTC-per-slave configuration and must be enforced as a compile-time configuration check in both endpoint implementations.

The STM32 sends STATUS and a complete PACK/CELL/TEMPERATURE snapshot when fresh measurement data is available, no more often than once every 500 ms. It sends EVENT immediately on a change. The Gateway must treat PACK, CELL, and TEMPERATURE data as invalid while STATUS says measurements are not fresh.

3.3.1.4. States, faults, and events

BMS states are 0 DEVICE_INITIALIZATION, 1 REGISTER_INITIALIZATION, 2 PERFORMING_DIAGNOSTICS, 3 RUNNING, and 4 PANIC. HV states are 0 OFF, 1 SELF_TEST, 2 PRECHARGE, 3 CONTACTOR_CLOSE, and 4 RUN.

BMS fault bitmap bits 0–15 are, in order: INVALID_CONFIG, TPL_FAULT, CID_INITIALIZATION_FAULT, REGISTER_INITIALIZATION_FAULT, CELL_BALANCING_FAULT, DIAGNOSTICS_FAULT, OVERVOLTAGE_LIMIT, UNDERVOLTAGE_LIMIT, TEMPERATURE_LIMIT, OVERCURRENT_LIMIT, IC_TEMPERATURE, SOC_LIMIT, OPEN_SHORT_FAULT, SYSTEM_FAULT, COMMUNICATION_TIMEOUT, and HV_SUPERVISOR_FAULT.

HV fault bitmap bits 0–6 are: SENSOR_DIAGNOSTIC, USB_ONLY, BATTERY_VOLTAGE_MISMATCH, LOAD_SIDE_ENERGIZED, PRECHARGE_TIMEOUT, PRECHARGE_VOLTAGE_LOST, and CONTACTOR_VOLTAGE_LOST. An active reason is present now; a latched reason remains a block after its live condition clears, until the STM32 accepts a supported clear procedure and revalidates the system.

EVENT is three bytes:

event_id:u8 | value:u16

ID	Event	Value
0x01	BMS state changed	new BMS state
0x02	HV state changed	new HV state
0x03	BMS active faults changed	new active mask
0x04	BMS latched faults changed	new latched mask
0x05	HV active reasons changed	new active mask
0x06	HV latched reasons changed	new latched mask
0x07	measurement freshness changed	0 or 1

EVENT is a convenience notification; STATUS is authoritative, so loss of an event is harmless.

3.3.1.5. Services

The service-request payload is service_id:u8 | arguments...; the service-response payload is service_id:u8 | result:u8 | response_data.... Result values are 0 OK, 1 DENIED, 2 INVALID, and 3 BUSY. INVALID covers an unknown service, bad request length, bad argument, or nonexistent slave index. DENIED means a valid request cannot safely be performed in the current STM32 state. BUSY means another named service is still in progress. OK means that the STM32 accepted and invoked the requested operation; STATUS and EVENT show the resulting operating state.

ID	Service	Request arguments	OK response data
0x01	GET_STATUS	none	17-byte STATUS
0x02	SET_RUN_REQUEST	requested:u8 (0 or 1)	none
0x03	CLEAR_FAULTS	none	none

ID	Service	Request arguments	OK response data
0x04	READ_REGISTER	slave_index:u8, register:u8	slave_index:u8, register:u8, value:u16
0x05	SET_RTC	unix_time_s:u32 UTC	none
0x06	GET_DEVICE_INFO	none	firmware_version:u32
0x07	ENTER_STM32_BOOTLOADER	firmware_version:u32, image_length:u32, image_crc32:u32	none
0x08	GET_RTC	none	unix_time_s:u32 UTC

SET_RUN_REQUEST(0) immediately removes the request through the STM32 PCC path. A request of 1 does not guarantee an HV start: the STM32 alone decides whether BMS and HV checks permit the sequence. CLEAR_FAULTS is denied while run is requested or live HV conditions are unhealthy. Neither the Gateway nor Home Assistant can bypass a fault, write BCC registers, or override the HV supervisor. `firmware_version` is packed as `major | (minor << 8) | (patch << 16) | (build << 24)`.

ENTER_STM32_BOOTLOADER is the only STM32-update operation in this protocol. The Gateway must already have staged and CRC-checked the image. The STM32 validates the request shape and image length; `firmware_version` and `image_crc32` identify the Gateway-staged image but cannot be verified by an STM32 that does not receive its bytes. It returns DENIED unless the run request is off and it can de-energise the HV path and inhibit normal services. After OK, the STM32 drains that response, stops framed UART traffic, and enters the STM32 ROM bootloader. The Gateway then performs the ROM bootloader sync, transfer, readback verification, and Go command directly on USART1; these are not FlexBMS UART messages. On return to the application, the Gateway waits for heartbeat and uses GET_DEVICE_INFO to confirm the expected firmware version. There are deliberately no update-data, acknowledgement, retry, or status frame types.

3.3.1.6. Test vectors

The following complete frames are hexadecimal, including the little-endian CRC:

HEARTBEAT

46 42 01 01 00 00 00 8F 7A FB 7D

GET_STATUS, sequence 0x2A

46 42 01 10 2A 01 00 01 8F 21 B2 4B

SET_RUN_REQUEST(true), sequence 0x2B

46 42 01 10 2B 02 00 02 01 16 26 B1 DC

READ_REGISTER response, sequence 0x2C, slave 3, register 0x20, value 0x1234

46 42 01 11 2C 06 00 04 00 03 20 34 12 1A 45 F6 9C

ENTER_STM32_BOOTLOADER, sequence 0x2D, version 1.2.3 build 4, image length 131072 bytes, image CRC32 0xA1B2C3D4

46 42 01 10 2D 0D 00 07 01 02 03 04 00 00 02 00 D4 C3 B2 A1 42 C9 9E 95

3.3.2. Pinout

The Gateway uses an ESP32-C3-WROOM-02U-N4. The module pin names and numbers below follow the Espressif datasheet; net names follow the Master schematic. GPIO17 is used internally for module flash and is not exposed.

Module pin	GPIO / module name	Net	Use
1	3V3	p3v3_INV	3.3 V supply
2	EN	EN	Chip enable
3	GPIO4 / IO4	CAN_TX	CAN controller transmit
4	GPIO5 / IO5	CAN_RX	CAN controller receive
5	GPIO6 / IO6	RS485_R0	RS-485 receiver output
6	GPIO7 / IO7	RS485_DI	RS-485 driver input
7	GPIO8 / IO8	I08/STRAP	Strapping pin
8	GPIO9 / IO9	I09/B00T	Boot-mode strap

Module pin	GPIO / module name	Net	Use
9; EPAD 19	GND	GND_INV	Ground and module ground pad
10	GPIO10 / IO10	ESP_VBUS_SENSE	USB VBUS sense
11	GPIO20 / RXD	–	Not connected; UART0 RX unused
12	GPIO21 / TXD	–	Not connected; UART0 TX unused
13	GPIO18 / IO18	USB_ESP32_N	USB D-
14	GPIO19 / IO19	USB_ESP32_P	USB D+
15	GPIO3 / IO3	UART_ESP_RX	STM32 USART1 TX input
16	GPIO2 / IO2	UART_ESP_TX	STM32 USART1 RX output; strapping pin
17	GPIO1 / IO1	USR_LED	Gateway user/status LED
18	GPIO0 / IO0	RS485_EN	RS-485 driver enable

3.4. Energy management system

PLANNED

The EMS is the supervisory optimization layer. It should combine battery availability with site demand, PV production, inverter limits, dynamic electricity prices, EV charging, and other flexible loads.

The EMS may request charging or discharging, but it must operate inside limits supplied by the BMS and inverter. It must tolerate missing telemetry and fall back to a safe, predictable operating policy.

The EMS implementation and deployment location are not yet defined in this repository. Its first interface should be a small, explicit contract containing battery state, available charge/discharge power, requested power, data freshness, and reason/status information.

3.5. Home Assistant integration

PLANNED

Home Assistant is the intended user-facing integration point for the installation. It can provide:

- battery, inverter, PV, grid, and load dashboards;
- fault and availability notifications;
- manual operating-mode selection;
- automation around dynamic prices and flexible loads; and
- long-term energy and temperature history.

Home Assistant must not become a safety dependency. If Home Assistant is unavailable, the STM32 must continue to protect the battery and the EMS must remain within the last valid limits or stop requesting power.

The entity model, discovery mechanism, command permissions, and dashboard design are still open.

3.6. Companion application

CURRENT

The current in-tree `Software/bms_companion` is a legacy copy and is not the source for Home-BESS work. `Software/Companion` is the FlexBMS-owned BMS-only application with two web targets: a direct USB Web Serial target for independent service diagnostics and an ESP32-hosted WebSocket target for local maintenance. A portable, unsigned Windows executable packages the direct-USB target in a thin Electron/Chromium wrapper. It has no installer, updater, or native serial-protocol implementation.

Both targets provide stale-status explanation, live fresh telemetry, CSV logging, named service requests, browser-time RTC synchronisation with device-time readback, and read-only register inspection. The Gateway target also shows Wi-Fi and UART state. Neither target exposes a raw terminal or firmware-update UI; Gateway requests use an explicit allowlist.

`scripts/build-release.ps1` interactively confirms the release version, updates the Companion package version when a new one is entered, and builds the Companion, Gateway, and STM32 artifacts with checksums. `-Version`

supports unattended use. Each release includes `flash-release.ps1`: ST-Link/SWD flashes the STM32 image, while a selected ESP32-C3 COM port flashes a complete Gateway factory image. Both actions start immediately after checksum verification. Repeating an existing version requires an explicit choice and creates a timestamped output without overwriting the prior release.

4. Operating concept

4.1. Start-up

At start-up, the STM32 validates its configuration and initializes the complete TPL chain. It assigns each battery monitor a chain address, configures its registers, runs diagnostics, and obtains a fresh measurement set. Both HV outputs remain off. After BMS validation, the separate HV supervisor can run its self-test and precharge sequence only when a new run request is supplied.

4.2. Normal operation

During normal operation the STM32 repeatedly measures the pack and evaluates its safety state. Battery limits and status are made available to the surrounding system. The inverter converts energy between the DC battery and the AC installation, while the future EMS decides the desired operating point.

4.3. Faulted operation

A battery, measurement, or communication fault immediately removes relay permission. Monitoring continues where possible so that the live condition can be observed. Faults remain latched until they are acknowledged through the supported service path and the BMS has successfully revalidated the battery chain.

The BMS exposes the HV supervisor as one aggregate blocking fault while the supervisor retains separate active, latched, and historical HV reasons. The Companion clear command can clear an HV latch only when the live HV conditions are healthy. The BMS then repeats chain initialization, diagnostics, and a complete fault-checked measurement cycle before the latch is released. If the retained run request is true, the BMS may then resume its normal startup sequence. Historical reasons remain available until reboot.

Configuration and initial TPL setup faults require a reboot. The ESP32, EMS, and Home Assistant cannot override a blocking STM32 fault.

5. Interfaces and ownership

Information or action	Owner	Consumer
Cell voltages and temperatures	STM32 BMS	Diagnostics, inverter limits, telemetry
Battery current and SoC	STM32 BMS	Inverter, EMS, Home Assistant
Battery safety limits	STM32 BMS	Inverter and EMS
Contactor and precharge	STM32 BMS	Physical HV path
Inverter operating state	Inverter	ESP32 / EMS / Home Assistant
Requested charge/discharge power	EMS	Inverter-facing control interface
Dashboards and automation	Home Assistant	User and household services

The protocol between these layers should carry explicit validity and freshness information. A stale value must never be interpreted as a current permission to charge, discharge, or close the contactor.

6. Current project structure

Repository area	Contents
Hardware/Master	Master schematic, PCB, harnesses, and production data.
Hardware/ModuleBoard	Battery module-board schematic, PCB, and production data.
Software/Master	STM32G491 firmware and platform configuration.
Software/bms_companion	Legacy in-tree Companion copy; not a Home-BESS source of truth.
Software/Companion	Current FlexBMS BMS maintenance UI with direct USB, Gateway, and portable Windows package outputs.
Software/Gateway	ESP32 UART v1, Wi-Fi provisioning/recovery, and compiled Companion serving; MQTT, OTA, and CAN work remain planned.
Documentation/protocol/uart-v1.md	Canonical framed BMS protocol for Gateway UART and direct USB CDC, including test vectors.
Simulations	Electrical simulation files used during hardware development.
Documentation	This system overview and its Typst build setup.

The repository contains the initial ESP32 Gateway and FlexBMS Companion implementations. EMS and Home Assistant remain planned.

7. Open topics

This overview is intentionally incomplete. The next useful documentation additions are:

- confirmed as-built module, NTC, and high-voltage topology;
- final inverter selection, protocol compatibility, and commissioning sequence;
- system-level operating, fault, recovery, and shutdown policy;
- commissioning of the ESP32 telemetry, local-network, and update architecture; and
- EMS and Home Assistant interfaces, fallback behavior, and user permissions.